SentinelAI: 4-Team Parallel Integration Master Strategy

Cognizant NPN 2027 Cybersecurity Hackathon — Activity 4

---

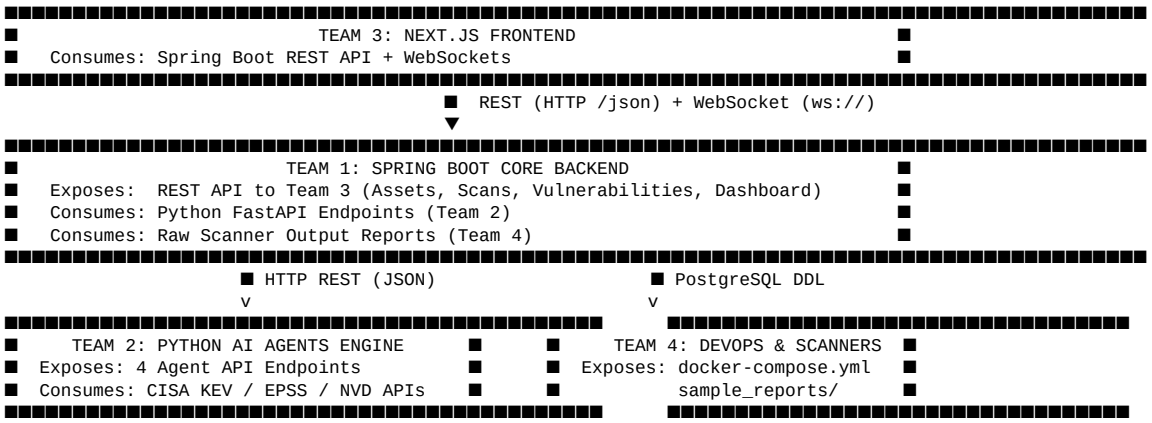
1. 4-Team Division & Responsibility Matrix

To enable 8 developers (2 per team) to work in parallel without blocking each other, the platform is partitioned into 4 decoupled, domain-bounded teams:

Team	Name	Primary Domain	Core Stack
Team 1	**Core Backend & Data**	System Gateway, RBAC, Asset/Scan Management, DB Persistence, WebSocket Server	Java 17, Spring Boot 3, PostgreSQL 16, Spring Security, JPA
Team 2	**AI Engine & Threat Intel**	4 AI Agents (Parsing, Noise Reduction, EPSS/KEV Enrichment, Risk Scoring)	Python 3.11, FastAPI, pandas, XGBoost, httpx
Team 3	**Security Dashboard & UI**	SentinelAI UI, Flow View Canvas, Risk Tables, Timeline Stream, Analytics	Next.js 14, React 18, Anime.js, Tailwind CSS
Team 4	**DevOps, Scanners & E2E**	Multi-Scanner Sandbox, Sample Ingestion, Docker Compose, GitHub Actions, E2E Testing	Docker, Nmap, Nuclei, ZAP, OpenVAS, GitHub Actions

---

2. Team Interfaces & Contracts (What Each Team Exposes)



Team 1 (Core Backend) Exposes:

- `POST /api/auth/login` → Authenticates users, returns JWT.
- `POST /api/assets` & `GET /api/assets` → Asset registration & inventory listing.
- `POST /api/scans` → Initiates multi-scanner pipeline.
- `GET /api/vulnerabilities` → Returns paginated, prioritized canonical findings.

• `GET /api/dashboard` → Summary metrics (`security_score`, `total_raw_findings`, `noise_reduction_pct`).

• `ws://localhost:8080/ws/pipeline` → Real-time status update stream for agent execution visualizer.

Team 2 (AI Engine) Exposes:

• `POST /api/v1/agent1/parse` → Ingests raw scanner reports (ZAP, Nuclei, Nmap, OpenVAS) → Returns `UnifiedFinding[]`.

• `POST /api/v1/agent2/reduce-noise` → Ingests `UnifiedFinding[]` → Returns deduplicated `CanonicalFinding[]` with `false_positive_prob`.

• `POST /api/v1/agent3/enrich` → Ingests `CanonicalFinding[]` → Enriches with CISA KEV, EPSS, NVD, Exploit-DB metrics.

• `POST /api/v1/agent4/score-and-ticket` → Ingests enriched findings → Calculates 0–100 Composite Score, SLA deadline, AI rationale text, and dispatches GitHub Issue.

Team 3 (Frontend UI) Exposes:

• Responsive Next.js Web Dashboard running at `http://localhost:3000`.

• Modular UI components (`FlowViewCanvas`, `MetricsGauge`, `LiveTimelineStream`, `PrioritizedVulnerabilityTable`).

Team 4 (DevOps & Scanners) Exposes:

• ephemereal `docker-compose.yml` environment linking Postgres, Python Agents, Spring Boot, and Next.js.

• Standardized test dataset directory (`sample_reports/nmap_scan.xml`, `zap_scan.json`, `nuclei_scan.jsonl`, `openvas_scan.xml`).

• Automated E2E verification test harness (`scripts/test_e2e_pipeline.sh`).

---

3. Strict Contract Standards & Schemas

Standard 1: `UnifiedFinding` Schema (Team 4 → Team 2 → Team 1)

```
{
  "raw_finding_id": "string (UUID)",
  "scanner_name": "NMAP | NUCLEI | OWASP_ZAP | OPENVAS",
  "target_host": "string (IP or Hostname)",
  "target_port": "integer",
  "target_path": "string",
  "cve_id": "string (CVE-YYYY-NNNN or null)",
  "cwe_id": "string (CWE-NNN or null)",
  "title": "string",
  "description": "string",
  "raw_severity": "CRITICAL | HIGH | MEDIUM | LOW | INFO",
  "cvss_base_score": "float (0.0 to 10.0)",
  "evidence": "string",
  "confidence": "HIGH | MEDIUM | LOW"
}
```

Standard 2: `CanonicalFinding` Schema (Team 2 → Team 1)

```
{
  "finding_id": "string (UUID)",
  "fingerprint_hash": "string (MD5)",
  "cve_id": "string",
  "vulnerability_name": "string",
  "target_host": "string",
  "target_port": "integer",
  "scanner_sources": ["NMAP", "NUCLEI", "OWASP_ZAP"],
  "false_positive_prob": "float (0.0 to 1.0)",
  "is_suppressed": "boolean",
  "is_accepted_risk": "boolean",
```

```
  "is_cisa_kev": "boolean",
  "epss_score": "float (0.0 to 1.0)",
  "composite_risk_score": "float (0.0 to 100.0)",
  "priority_level": "P0_CRITICAL | P1_HIGH | P2_MEDIUM | P3_LOW",
  "sla_deadline": "ISO-8601 Timestamp",
  "explainable_rationale": "string"
}
```

---

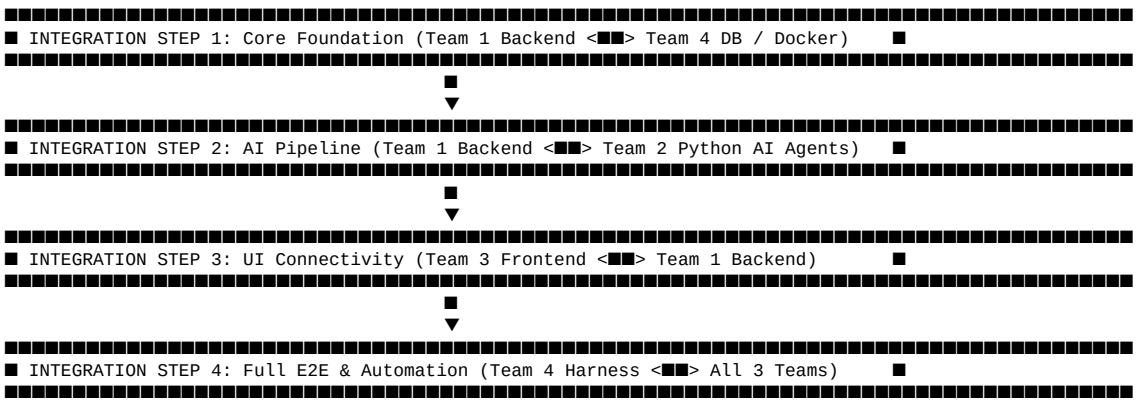
4. Independent Development Strategy (Mocking & Stubbing)

To ensure zero downtime, teams use explicit mock servers during early development:

Team	What They Build	What They Mock During Initial Dev	How It Is Mocked
Team 1 (Backend)	DB schema, JWT auth, Asset/Scan controllers	Team 2 Python AI Agents	`MockAgentClient.java` returning static JSON arrays from `src/test/resources/mocks/`
Team 2 (AI Engine)	4 AI FastAPI Agent algorithms & XGBoost	Live CISA/EPSS APIs & Real Scanner Scans	Local JSON fixtures (`mock_kev.json`, `mock_epss.json`) & static FastAPI router responses
Team 3 (Frontend)	Next.js Dashboard & Flow View Canvas	Spring Boot REST API & WebSockets	`MSW` (Mock Service Worker) or `json-server` running mock endpoints on `:8080`
Team 4 (DevOps)	Docker compose, scanner scripts, CI/CD	Fully completed app services	Ephemeral base containers with health checks & dummy echo servers

---

5. Integration Sequence (4 Chronological Milestones)



Integration Step 1: Backend + Database (Team 1 + Team 4)

• **Goal**: Connect Spring Boot to PostgreSQL running in Docker Compose.

• **Action**: Team 4 provides `docker-compose.yml` containing PostgreSQL. Team 1 configures `application.yml` and runs JPA DDL schema migrations.

• **Verification**: `curl http://localhost:8080/actuator/health` returns `{"status": "UP", "components": {"db": {"status": "UP"}}}`.

Integration Step 2: Backend + AI Engine Pipeline (Team 1 + Team 2)

• **Goal**: Connect Spring Boot orchestrator to Python FastAPI AI agent microservices.

• **Action**: Team 1 replaces `MockAgentClient.java` with real `RestClient` calls to Team 2's FastAPI endpoints on `http://python-agents:8000`. Team 1 passes sample reports from Team 4 (`nmap_scan.xml`, `zap_scan.json`) through Agents 1 → 2 → 3 → 4.

• **Verification**: Passing 10 raw findings returns clean deduplicated records with composite risk scores > 0 in PostgreSQL.

Integration Step 3: Frontend + Backend Integration (Team 3 + Team 1)

• **Goal**: Wire Next.js dashboard UI to Spring Boot REST API & WebSocket notifications.

• **Action**: Team 3 disables Mock Service Worker (MSW) and points `NEXT_PUBLIC_API_URL` to `http://localhost:8080`. Team 1 enables CORS headers for `http://localhost:3000`.

• **Verification**: Triggering a scan in Next.js UI updates the Live Timeline stream and renders canonical vulnerabilities in the Flow View & Risk Table.

Integration Step 4: Full System E2E & Multi-Scanner Automation (Team 4 + All)

• **Goal**: Execute complete workflow from scanner report ingestion to live GitHub Issue ticketing.

• **Action**: Team 4 runs `test_e2e_pipeline.sh` inside Docker Compose, feeding 2,500 raw findings.

• **Verification**:

1. Raw alerts (2,500) ingested.

2. Deduplication & noise reduction reduces count to 15 canonical findings (**94% noise reduction**).

3. GitHub repository receives live P0 Issue with SLA deadline.

4. Next.js dashboard displays Security Score `96/100`.

---

6. Git Branching & Code Merging Strategy

Monorepo Structure

```
sentinelai/
■■■ .github/workflows/ci-cd.yml
■■■ backend/              (Team 1 Ownership)
■■■ agents_service/       (Team 2 Ownership)
■■■ frontend/             (Team 3 Ownership)
■■■ database/             (Team 1 & 4 Ownership)
■■■ sample_reports/       (Team 4 Ownership)
■■■ docker-compose.yml    (Team 4 Ownership)
```

Branching Rules

• `main` → Protected production branch. Code must compile and pass all tests.

• `develop` → Integration branch where 4 teams merge feature branches.

• Feature Branches:

• Team 1: `feature/backend-asset-api`, `feature/backend-agent-client`

• Team 2: `feature/agent1-zap-parser`, `feature/agent2-xgboost-dedup`

• Team 3: `feature/ui-flow-view`, `feature/ui-risk-table`

• Team 4: `feature/docker-compose-setup`, `feature/e2e-test-harness`

Pull Request (PR) Policy

• Every PR targeting `develop` requires **at least 1 approval** from another team lead.

• GitHub Actions automatically runs automated checks:

1. Backend Maven build (`mvn clean test`).

2. Python `pytest` suite for AI agents.

3. Frontend Next.js build (`npm run build`).

4. Docker Compose build test (`docker-compose build`).

---

7. Complete End-to-End Execution Flow (From Click to GitHub Ticket)

```
1. USER ACTION (Next.js Dashboard - Team 3)
   Analyst clicks "Start Multi-Scanner Pipeline" on Target: api.acmebank.com
   ■
   ▼
2. API REQUEST (Spring Boot Backend - Team 1)
   Next.js sends POST /api/scans -> Backend verifies asset authorization in PostgreSQL
   ■
   ▼
3. RAW REPORT INGESTION (Team 4 Sample Reports -> Team 1)
   Backend reads sample ZAP, Nuclei, Nmap reports from volume mount
   ■
   ▼
4. AGENT 1 EXECUTION (Python FastAPI - Team 2)
   Spring Boot calls POST /api/v1/agent1/parse -> Agent 1 parses XML/JSON into UnifiedFinding[]
   ■
   ▼
5. AGENT 2 NOISE REDUCTION (Python FastAPI - Team 2)
   Backend calls POST /api/v1/agent2/reduce-noise -> Agent 2 calculates MD5 hash, pandas merges
   duplicates across ZAP & Nuclei, XGBoost filters false positives (Noise reduced by 94%)
   ■
   ▼
6. AGENT 3 THREAT INTEL ENRICHMENT (Python FastAPI - Team 2)
   Backend calls POST /api/v1/agent3/enrich -> Agent 3 checks CISA KEV (Flagged=True), queries
   FIRST.org EPSS API (EPSS=0.972), fetches NVD CVSS v3.1 vector
   ■
   ▼
7. AGENT 4 RISK SCORING & TICKETING (Python FastAPI -> Team 1 -> GitHub)
   Backend calls POST /api/v1/agent4/score-and-ticket -> Agent 4 calculates Composite Score (92.4),
   assigns P0 Critical priority (24h SLA), generates Explainable AI text, dispatches GitHub Issue
   ■
   ▼
8. REAL-TIME UI UPDATE (Spring Boot -> Next.js - Team 3)
   Spring Boot broadcasts WebSocket event -> Next.js Live Timeline updates -> Flow View network node
   flashes Red -> Risk Table displays #1 Ranked Log4Shell vulnerability ready for remediation.
```

---

8. Summary Checklist for Team Leads

■ **Team 1 Lead**: Freeze REST API DTOs and database DDL script by End of Day 1. Provide Mock REST endpoints for Team 3.

■ **Team 2 Lead**: Freeze Pydantic JSON request/response schemas for all 4 Agents by End of Day 1. Deploy local FastAPI server on port 8000.

■ **Team 3 Lead**: Setup Next.js 14 skeleton with Mock Service Worker (MSW) matching Team 1's API contract by End of Day 1.

■ **Team 4 Lead**: Commit `docker-compose.yml`, base PostgreSQL database container, and standard `sample_reports/` by End of Day 1.

■ **All Leads**: Execute Step-by-Step Integration Sequence (Steps 1–4) sequentially on Days 2 & 3.